

DataEng S25: Project Assignment 3

Data Integration

Congratulations! By now you have a working, end-to-end data pipeline. Unfortunately, it does not have enough data to properly implement our Data Scientist's visualization. To fill out information such as "route ID" you need to access another source of data and build a new pipeline to integrate it with your initial pipeline. Here are your steps:

- A. access the stop event data
- B. build a new pipeline for the stop event data
- C. integrate the stop event data with the breadcrumb data
- D. testing

A. Stop Events Data

Access TriMet "Stop Events" data at this URL:

```
https://busdata.cs.pdx.edu/api/getStopEvents?vehicle_num=<vehicle_num>
```

As with the previous data source, this data set gives all TriMet vehicle stop events for a single day of operation. Again, make sure to replace the vehicle_num with the vehicle id's assigned to you.

B. New Pipeline

Your job is to build a new pipeline that operates just like the previous one, including use of Cloud Pub/Sub, automation, validation and loading.

In addition to the above, this new pipeline will require you to use classes, functions, and follow some basic OOP principles and design patterns. Do not use global variables. The objective here is to demonstrate how, with some refactoring and restructuring, your code becomes highly modular and flexible.

C. Integrate Stop Events with Bread Crumbs

Once the second pipeline is up and running, you will have two separate data streams flowing into your database: the BreadCrumb pipeline should be updating the values in the Breadcrumb table and the StopEvent pipeline should be updating the values in the Trip table. All columns for both tables must be filled with data correctly.

It will now be possible for your Visualizer component to pull from both tables and integrate the data to display it.

D. Visualization

[MapboxGL](#) is a data visualization tool that allows you to view your breadcrumb data and display it on a map. Your job is to integrate this tool with your database tables so that you can query the breadcrumb and trip data in your database server, transform to geoJSON format and display the resulting map visualization. To get started, [see this guide](#).

Alternatively, you may use an alternative visualization tool (such as folium) to create the required visualizations. We do not provide any guides for doing it, but you are free to do so if you prefer. The submitted visualizations must be equivalent or superior to the visualizations produced by the provided MapboxGL based visualization tool.

DataEng Project Assignment 3 Submission Document

Your Code

1. Provide a reference to the repository where you store your python code. If you are keeping it private then share it with the Professor and TA.
2. Put the code you want graded into a folder in your repository with the same name for the part you're submitting.
 - a. **If part 1, then the folder would be 'part1' or 'milestone1', and so on for each part.**
3. Include all code relevant to the pipeline for that step too. Part 2 would be all code from part 1 and then all additional code with part 2.

Continued on next page

Construct a table showing each day for which your pipeline successfully, automatically processed one complete day's worth of sensor readings **for breadcrumb data only**. The table should look like this:

Date	Day of Week	# breadcrumb records	# rows added to BreadCrumb table	# rows added to Trip table

Perform These Queries on Your DB and Report the Results

Query #1:

```
select tstamp::date, count(*)  
from breadcrumb  
group by tstamp::date  
order by tstamp::date ASC;
```

Query #2:

```
SELECT
    b.tstamp::date AS trip_date,
    COUNT(DISTINCT t.vehicle_id) AS num_vehicles,
    COUNT(DISTINCT t.trip_id) AS num_trips,
    COUNT(DISTINCT t.route_id) AS num_routes,
    COUNT(*) AS num_breadcrumbs
FROM
    Breadcrumb b
JOIN
    Trip t ON b.trip_id = t.trip_id
WHERE
    b.tstamp::date > '2023-01-08'
GROUP BY
    b.tstamp::date
ORDER BY
    trip_date ASC;
```

Make a copy of this document and update it to include the following visualizations. For each visualization extract from your database a list of (latitude, longitude, speed) tuples and then use the provided visualization code (see Section D above) to display bus speeds at all of the corresponding geographic coordinates. So, for example, if you are asked to visualize a “trip”, then you must query your database to find all of the (latitude, longitude, speed) tuples for that trip, and then display a map showing the recorded/calculated bus speed at each (latitude,longitude) location.

Display only trips and breadcrumbs from dates after January 8, 2023.

No need to produce software that neatly displays trips, routes, dates, times, etc. onto the visualization itself. Instead, just paste a screen capture of the map-based speed visualization into your submission document and then include a text description of the contents of the visualization. For example, text like this: *“Bus Speeds for all trips that traveled within 1km of Ladd Circle Park before 11am on Wednesday, February 15, 2023”*

Visualization 1. A visualization of speeds for a single trip for any bus that crossed the Tilikum Crossing Bridge. You choose the day, time and route for your selected trip. To find a trip that traverses this bridge, consider finding a trip that includes breadcrumbs within the rectangle bounded by (45.505621, -122.668632), and (45.504416, -122.665738).

Visualization 2. Any single trip on route 20 (you choose which trip) between the hours of 4pm and 6pm (you choose which date).

Visualization 3. All trips that traveled to and from PSU campus on any single weekday morning (you choose which date) between 9am and 11am.

Visualization 4. All trips that traveled within 1km of Ladd Circle Park (45.508537, -122.649434) on any selected morning before 11am (you choose which date).

Visualization 5a, 5b, 5c, Three or more additional visualizations of your choice. Indicate why you chose each particular visualization.